

Assignment #1 — Builder Pattern

Student: Tursynbekov Anuar

Program: Software Engineering

Course: Software Design Patterns

1. Introduction

For this assignment, I chose a **Car Construction System** as the product for implementing the Builder Creational Design Pattern.

The main product is a Car object that contains several parameters: engine, color, wheels, and number of seats. Different types of cars can require different configurations. For example, a sports car can have a V6 engine, sport wheels, and two seats, while a family car can have a V4 engine, standard wheels, and five seats. The Builder Pattern is suitable for this project because the Car object contains several parameters and can have different configurations. Instead of creating a car using a large constructor with many parameters, the object can be constructed step by step.

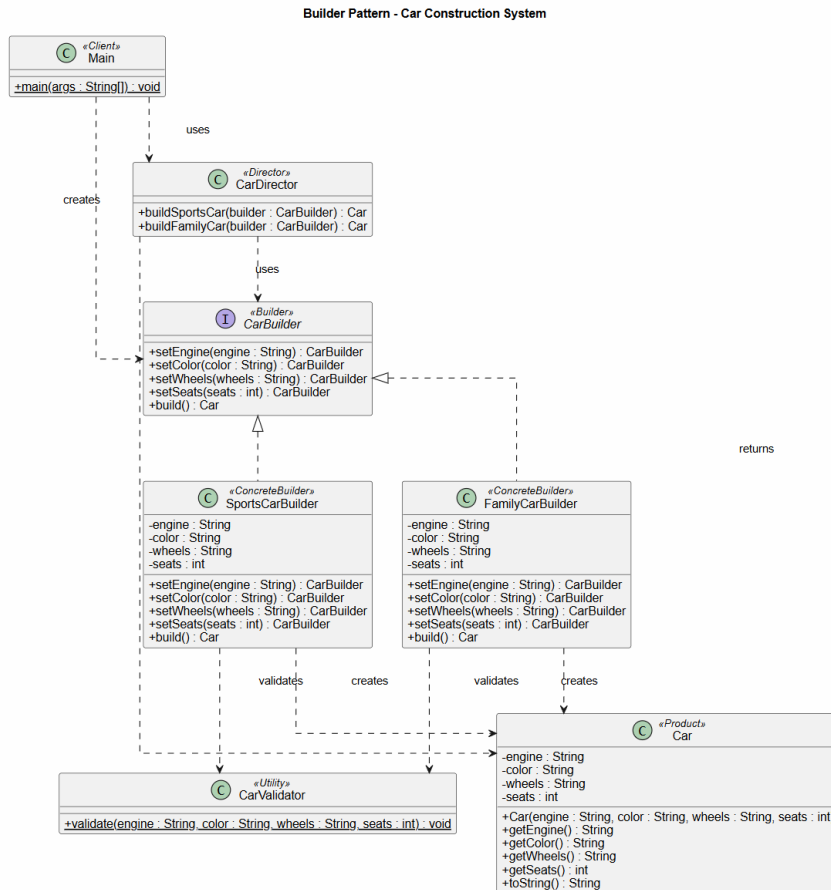
The implementation separates the construction process from the final Car object. The project uses a CarBuilder interface, two Concrete Builders (SportsCarBuilder and FamilyCarBuilder), and a CarDirector that controls reusable construction sequences.

The main components of the implementation are:

- Car — Product;
- CarBuilder — Builder;
- SportsCarBuilder — ConcreteBuilder;
- FamilyCarBuilder — ConcreteBuilder;
- CarDirector — Director;
- Main — Client;
- CarValidator — auxiliary validation class.

This approach makes the construction process more readable, flexible, and easier to maintain.

2. UML Class Diagram



The UML class diagram represents the structure of the implemented Builder Pattern.

The Car class is the **Product** because it represents the final object created by the builders.

The CarBuilder interface is the **Builder**. It defines the construction methods:

- setEngine();
- setColor();
- setWheels();
- setSeats();
- build().

SportsCarBuilder and FamilyCarBuilder are **Concrete Builders** because they implement the CarBuilder interface and construct Car objects.

CarDirector is the **Director**. It controls the sequence of construction steps and provides predefined configurations for sports and family cars.

Main is the **Client** because it uses the builders and Director to create the final objects.

CarValidator is an additional utility class responsible for checking whether the provided car parameters are valid.

[INSERT UML CLASS DIAGRAM HERE]

3. Clean Code Principles

3.1 Meaningful Names

The first Clean Code principle used in the project is **meaningful naming**.

Classes, methods, and variables have names that clearly describe their purpose. For example:

```
public Car buildSportsCar(CarBuilder builder) {
```

The method name `buildSportsCar` clearly shows that the method creates a sports car.

The Builder interface also uses descriptive method names:

```
CarBuilder setEngine(String engine);
```

```
CarBuilder setColor(String color);
```

```
CarBuilder setWheels(String wheels);
```

```
CarBuilder setSeats(int seats);
```

```
Car build();
```

These names make the code easier to understand without requiring additional comments.

3.2 Single Responsibility Principle

The second principle is **Single Responsibility**.

Each class in the project has a specific responsibility.

The Car class represents the final product.

SportsCarBuilder and FamilyCarBuilder are responsible for constructing cars.

CarDirector controls the construction sequence.

CarValidator is responsible for validation.

Main demonstrates how the system works.

For example, validation is not placed inside the Car class. Instead, it is handled by the separate CarValidator class.

This separation makes the project easier to maintain and modify.

3.3 DRY — Don't Repeat Yourself

The third principle is **DRY (Don't Repeat Yourself)**.

Both Concrete Builders need to validate the car parameters before creating a Car.

Instead of writing the same validation conditions in both builders, the project uses a separate CarValidator class.

The builders call:

```
CarValidator.validate(engine, color, wheels, seats);
```

The validation logic is therefore stored in one place.

For example:

```
if (engine == null || engine.isBlank()) {  
    throw new IllegalStateException("Engine is required");  
}
```

If the validation rules need to be changed, they only need to be modified inside CarValidator.

This prevents duplicated logic and makes future maintenance easier.

3.4 Small and Focused Methods

The fourth principle is using **small and focused methods**.

The methods in the project are designed to perform specific tasks.

For example, the build() method is responsible for validating the data and creating the final Car object:

```
@Override  
public Car build() {  
    CarValidator.validate(engine, color, wheels, seats);  
  
    return new Car(engine, color, wheels, seats);  
}
```

The method does not contain unrelated operations.

The Director methods are also focused on one task. For example, buildSportsCar() only defines the steps required to construct a sports car.

This makes the code easier to read and understand.

3.5 Fail Fast and Clear Validation

The fifth Clean Code principle is **Fail Fast with clear error handling**.

The application checks the data before creating the final object.

For example, the engine cannot be empty:

```
if (engine == null || engine.isBlank()) {  
    throw new IllegalStateException("Engine is required");  
}
```

The project also checks the color and wheels:

```
if (color == null || color.isBlank()) {  
    throw new IllegalStateException("Color is required");  
}
```

```
if (wheels == null || wheels.isBlank()) {  
    throw new IllegalStateException("Wheels are required");  
}
```

The number of seats must also be greater than zero:

```
if (seats <= 0) {  
    throw new IllegalStateException("Seats must be greater than 0");  
}
```

If invalid data is provided, the program immediately stops the construction process and provides a clear error message.

4. Conclusion

During this assignment, I implemented the Builder Creational Design Pattern using Java and applied it to a car construction system.

The main advantage I personally encountered was that the construction process became much more readable. Instead of passing several parameters directly to a large constructor, the car can be created step by step:

```
builder  
    .setEngine("V6")  
    .setColor("Red")  
    .setWheels("Sport")  
    .setSeats(2)  
    .build();
```

Another advantage was the ability to create different configurations of the same Car product. The project supports both sports and family car configurations. The CarDirector also makes predefined construction sequences reusable. This reduces the amount of construction logic that needs to be written in the client code. However, I also encountered a disadvantage. The Builder Pattern requires more classes and more code than simply creating an object with a constructor. For a simple object with only a few parameters, using a Builder could make the project unnecessarily complicated.

Another challenge was separating the responsibilities correctly between the Product, Builder, Concrete Builders, Director, and validation utility.

Overall, I found the Builder Pattern useful for objects that have multiple parameters and different possible configurations. The implementation also helped me understand how design patterns can improve code structure and maintainability.

5. GitHub Repository

The complete source code and project history are available in my GitHub repository.

GitHub Repository:

<https://github.com/tursynbekovanuar1-design/Builder-Pattern/tree/main>

The repository contains the Java implementation of the Builder Pattern, including the Product, Builder, Concrete Builders, Director, Client, validation functionality, and incremental commit history.